Quiz — 1.1.2 Types of Processor

Name: ___ Date: _ ***Mark:*** ___ / 25

OCR H446 · Component 01 · 1.1.2

Section A — Multiple choice (8 marks)

1. A key feature of the **Von Neumann** architecture is that it uses: A. Separate memories and buses for data and instructions B. A single shared memory and bus for both data and instructions C. Thousands of small cores for parallel processing D. Only read-only memory Your answer: ☐

2. Which architecture allows an instruction to be **fetched at the same time** as data is read or written? A. Von Neumann B. CISC C. Harvard D. Single-core Your answer: ☐

3. Compared with CISC, **RISC** processors typically have: A. Many complex, variable-length instructions B. Few simple, fixed-length instructions aiming for one cycle each C. All complexity built into the hardware D. Higher power consumption Your answer: ☐

4. In a **CISC** processor, the complexity of executing instructions is mainly handled by the: A. Compiler / software B. Hardware C. Cache D. Address bus Your answer: ☐

5. Which type of processor is most commonly found in **mobile phones and embedded devices** because of its low power use? A. CISC (x86) B. RISC (ARM) C. Von Neumann only D. GPU only Your answer: ☐

6. A **GPU** is best suited to: A. Highly sequential, branch-heavy tasks B. Applying the same operation to large blocks of data in parallel C. Running the operating system kernel D. Storing data permanently when the power is off Your answer: ☐

7. Which of the following is an example of **GPGPU** (general-purpose computing on a GPU)? A. Rendering pixels for a 3D game B. Training a neural network / machine learning C. Decoding a single instruction in the CIR D. Holding the BIOS firmware Your answer: ☐

8. A task will **not** benefit much from parallel processing when: A. It can be split into many independent sub-tasks B. Each step depends on the result of the previous step (data dependency) C. The software is written to use multiple threads D. There are many cores available Your answer: ☐

Section B — Short answer (12 marks)

9. State **three** differences between Von Neumann and Harvard architectures. **[3]**

10. Many modern CPUs are described as a **modified (hybrid) Harvard** architecture. Explain what this means. **[2]**

11. Explain **two** differences between CISC and RISC processors. **[2]**

12. Explain why a **GPU** can render 3D graphics so quickly. **[2]**

13. Define the term **multicore processor**. **[1]**

14. A program sorts a list and then searches it. The search cannot start until the sort has finished. Explain why splitting this work across multiple cores gives **limited** benefit. **[2]**

Section C — Exam-style question (5 marks)

15. A research team needs to run large-scale scientific simulations that apply the same mathematical calculation to millions of data points. They are deciding whether to invest in a powerful multicore **CPU** or a high-end **GPU**.

Discuss which processor type is better suited to this workload and justify your answer. **(AO2/AO3) [5]**

Answer key

Section A (8 marks — 1 each)

1. **B** — Von Neumann uses one shared memory and bus for both data and instructions.
2. **C** — Harvard has separate memories/buses, allowing simultaneous instruction fetch and data access.
3. **B** — RISC = few, simple, fixed-length instructions, aiming for one cycle each.
4. **B** — In CISC the complexity is in the hardware (microcode); in RISC it is in software/compiler.
5. **B** — RISC/ARM is used in mobile/embedded devices for low power consumption.
6. **B** — GPUs apply the same operation to large data blocks in parallel (data parallelism).
7. **B** — Training a neural network is a non-graphics (GPGPU) use; rendering pixels is standard graphics.
8. **B** — Data dependency (a step needs the previous step's result) prevents effective parallelisation.

Section B (12 marks)

9. **[3]** — 1 mark per valid difference (max 3). Examples: - Von Neumann = one shared memory; Harvard = separate memories for data and instructions. - Von Neumann = shared bus; Harvard = separate buses. - Von Neumann can suffer a bottleneck (cannot fetch instruction and access data at once); Harvard can do both simultaneously. - Von Neumann is simpler/cheaper/more flexible; Harvard is more complex/costly. - Von Neumann used in general-purpose PCs; Harvard used in embedded/microcontrollers/DSPs.

10. **[2]** — 1 mark: it uses a **single unified main memory** (Von Neumann style) for programs and data; 1 mark: but has **separate L1 instruction and data caches** (Harvard style) close to the core, giving some simultaneous-access benefit.

11. **[2]** — 1 mark per difference (max 2). Examples: CISC has many complex variable-length instructions vs RISC few simple fixed-length; complexity in hardware (CISC) vs software/compiler (RISC); RISC easier to pipeline; RISC lower power; CISC used in x86 desktops vs RISC in ARM mobile/embedded.

12. **[2]** — 1 mark: a GPU has **thousands of small cores**; 1 mark: it can process many pixels/vertices **in parallel** (the same operation on large data sets simultaneously), so rendering is much faster than doing it sequentially.

13. **[1]** — A processor (chip) containing **two or more independent cores**, each able to run its own fetch–decode–execute cycle, allowing simultaneous processing.

14. **[2]** — 1 mark: there is a **data dependency** — the search needs the completed (sorted) list, so it cannot begin until the sort finishes. 1 mark: the two stages must run **sequentially**, so extra cores cannot speed up the overall sequence (only the sort itself might be parallelised); benefit is limited by the **serial fraction**.

Section C (5 marks)

15. **[5]** — Point/level marking, up to 5 for a justified, applied discussion. Indicative content: - The workload applies the **same calculation to millions of data points** — this is highly **data-parallel**, with little branching or dependency between points. - A **GPU** has thousands of small cores and excels at performing the same operation on large data sets in parallel (data parallelism / GPGPU), so it can process far more data points simultaneously. - A multicore CPU has fewer, more powerful cores better suited to sequential, branch-heavy, dependency-laden tasks — fewer parallel lanes for this kind of work. - Caveat: the software must be written to use the GPU; very branchy or strongly dependent calculations would suit the CPU better. - Justified conclusion: for this uniform, massively data-parallel simulation the **GPU is the better choice** because it maximises throughput across many data points. Max 4 if no clear justified judgement is given.

Total: 8 + 12 + 5 = 25